LAPORAN PRAKTIKUM

- **Identitas Praktikum**

Nama MK	: Praktikum Pemrograman Berorientasi Objek
Kode MK	: PBO
Bobot SKS	: 1 SKS
Tempat	: Lab MM
Hari, tanggal	: Kamis, 09/04/2026
Jam	: -
Topik praktikum	: Modul-2

- **Identitas Mahasiswa**

Nama lengkap	: Naufal Labib Asyidiq
NIM	: 103112400108
Program Studi	: S-1 Teknik Informatika

- **Algoritma / studi kasus (jika ada) & Penjelasan**

Buat class Hero dengan atribut: nama (String) , hp (int) , mana (int) , damage (int), semua atribut harus bersifat private.
Memastikan setiap hero memiliki **nilai awal yang valid saat dibuat.**

Hp minimal 0 maksimal 100
Mana minimal 0 maksimal 100
Damage minimal 0 maksimal 15

Mengatur bagaimana hero bisa "menyerang" dengan ketentuan :

Saat seorang hero menyerang hero lain damage yang diberikan berasal dari nilai damage dasar hero tersebut
HP musuh akan berkurang sesuai damage
HP tidak boleh kurang dari 0

Mengatur bagaimana hero bisa "menggunakan skill" dengan ketentuan :

Skill adalah serangan khusus yang lebih kuat, tetapi Menggunakan sejumlah mana sebanyak 25
Damage yang diberikan lebih besar dari serangan biasa sebesar 2× damage

Mengatur bagaimana hero bisa "heal/regen" dengan ketentuan :

Menambah HP sebanyak 15
Jika HP sudah maksimum tidak bisa melakukan regen

Hero dapat menimpilkan status atribut dan ada keterangan DEAD ketika HP = 0 dan ALIVE ketika HP > 0

Tidak boleh ada setter untuk: hp,mana, damage sedangkan nama tidak boleh diubah setelah object dibuat

Algoritma

Sistem simulasi pertarungan Hero ini dibangun dengan prinsip *encapsulation* secara ketat untuk menjaga keamanan dan konsistensi data. Setiap karakter direpresentasikan oleh *class* Hero yang memiliki atribut privat berupa nama, *Health Point* (HP), *mana*, dan *damage* dasar. Saat objek karakter pertama kali diciptakan, nilai-nilai atribut ini akan melewati proses validasi menggunakan fungsi internal bernama clamp. Fungsi ini bertugas mengunci nilai agar tidak berada di luar batas yang diizinkan; contohnya, HP dan *mana* tidak boleh kurang dari 0 atau lebih dari 100, sedangkan *damage* dibatasi maksimal di angka 15. Dengan mekanisme ini, sistem menjamin bahwa tidak ada data *hero* yang tidak masuk akal (seperti HP negatif atau atribut berlebih) sejak awal permainan, sehingga *state* objek selalu valid.

Untuk interaksi antar karakter, sistem menyediakan tiga aksi utama yaitu serangan biasa (attack), serangan spesial (useSkill), dan pemulihan (regen), yang semuanya bergantung pada metode pengecekan status kehidupan (isAlive). Jika seorang *hero* sudah berstatus *DEAD* (HP mencapai 0), seluruh percobaan aksinya akan otomatis dibatalkan oleh sistem. Saat melakukan serangan biasa, HP target akan berkurang sesuai nilai *damage* penyerang, sementara serangan spesial menuntut pengorbanan 25 poin *mana* untuk dapat menghasilkan kerusakan sebesar dua kali lipat. Di sisi lain, aksi pemulihan memungkinkan *hero* untuk memulihkan 15 poin HP. Seluruh proses manipulasi angka HP ini diamankan lebih lanjut menggunakan fungsi Math.max dan Math.min untuk mencegah HP target menjadi negatif akibat besarnya kerusakan, atau HP penyerang melebihi kapasitas 100 saat melakukan *regen*.

- **Kode Program & Penjelasan**

  Class Hero

```
public class Hero {
    private String nama;
    private int hp;
    private int mana;
    private int damage;

    public Hero(String nama, int hp, int mana, int damage) {
        this.nama = nama;
        this.hp = clamp(hp, 0, 100);
        this.mana = clamp(mana, 0, 100);
        this.damage = clamp(damage, 0, 15);
    }
```

```
    private int clamp(int value, int min, int max) {
        if (value < min) {
            return min;
        }
        if (value > max) {
            return max;
        }
        return value;
    }

    public String getNama() {
        return nama;
    }

    public int getHp() {
        return hp;
    }

    public int getMana() {
        return mana;
    }

    public int getDamage() {
        return damage;
    }

    public boolean isAlive() {
        return hp > 0;
    }

    public void attack(Hero target) {
        if (!isAlive()) {
            System.out.println(nama + " sudah DEAD dan tidak bisa menyerang.");
            return;
        }
        if (target == null) {
            System.out.println("Target tidak valid.");
            return;
```

```
        }

        int oldHp = target.hp;
        target.hp = Math.max(0, target.hp - damage);

        System.out.println(nama + " menyerang " + target.nama
                + " sebesar " + damage + " damage."
                + " (HP " + target.nama + ":" + oldHp + " -> " + target.hp + ")");
    }

    public void useSkill(Hero target) {
        if (!isAlive()) {
            System.out.println(nama + " sudah DEAD dan tidak bisa menggunakan
skill.");
            return;
        }
        if (target == null) {
            System.out.println("Target tidak valid.");
            return;
        }

        final int manaCost = 25;
        if (mana < manaCost) {
            System.out.println(nama + " tidak punya mana cukup untuk skill.");
            return;
        }

        int skillDamage = damage * 2;
        mana -= manaCost;
        int oldHp = target.hp;
        target.hp = Math.max(0, target.hp - skillDamage);

        System.out.println(nama + " menggunakan skill ke " + target.nama
                + " sebesar " + skillDamage + " damage"
                + " (mana -" + manaCost + ")."
                + " (HP " + target.nama + ":" + oldHp + " -> " + target.hp + ")");
    }

    public void regen() {
```

```
        if (!isAlive()) {
            System.out.println(nama + " sudah DEAD dan tidak bisa regen.");
            return;
        }
        if (hp >= 100) {
            System.out.println(nama + " memiliki HP maksimum dan tidak bisa
regen.");
            return;
        }

        int oldHp = hp;
        hp = Math.min(100, hp + 15);

        System.out.println(nama + " melakukan regen +15 HP. (" + oldHp + " -> "
+ hp + ")");
    }

    public void showStatus() {
        String status = isAlive() ? "ALIVE" : "DEAD";
        System.out.println("Hero: " + nama);
        System.out.println("HP: " + hp + "/100");
        System.out.println("Mana: " + mana + "/100");
        System.out.println("Damage: " + damage);
        System.out.println("Status: " + status);
        System.out.println("-----------------");
    }
}
```

Class Main.java

```
public class Main {
    public static void main(String[] args) {
        Hero alucard = new Hero("Alucard", 95, 60, 12);
        Hero miya = new Hero("Miya", 80, 40, 10);
        Hero franco = new Hero("Franco", 120, -5, 20);

        System.out.println("=== STATUS AWAL ===");
        alucard.showStatus();
        miya.showStatus();
        franco.showStatus();
```

```
            System.out.println("=== SIMULASI BATTLE ===");
            alucard.attack(miya);
            miya.useSkill(alucard);
            franco.useSkill(miya);
            miya.regen();

            System.out.println("=== STATUS SETELAH AKSI ===");
            alucard.showStatus();
            miya.showStatus();
            franco.showStatus();

            System.out.println("=== UJI HERO DEAD TIDAK BISA MENYERANG
        ===");
            Hero zilong = new Hero("Zilong", 10, 100, 8);
            Hero layla = new Hero("Layla", 100, 100, 14);

            layla.useSkill(zilong);
            zilong.attack(layla);

            zilong.showStatus();
            layla.showStatus();
        }
}
```

Penejelasan kode

Program simulasi ini berpusat pada penerapan konsep *Encapsulation* (pembungkusan data) melalui *class* Hero yang bertindak sebagai cetak biru karakter. Semua atribut penting seperti nama, HP, *mana*, dan *damage* dikunci menggunakan *modifier private* agar tidak bisa dimanipulasi secara sembarangan oleh *file* atau sistem luar. Untuk menjamin bahwa setiap karakter memiliki status yang rasional sejak awal permainan, program mengandalkan metode *constructor* yang digabungkan dengan fungsi internal bernama clamp. Fungsi penjaga ini bertugas menyaring angka yang dimasukkan saat pembuatan karakter; jika angka berada di luar batas yang ditetapkan, seperti HP bernilai 120 atau *mana* bernilai -5, sistem akan memotong dan mengembalikannya ke batas yang sah (maksimal 100 dan minimal 0) sehingga *bug* atau eror data dapat dicegah.

Selain melindungi data statis, *class* Hero juga menjadi pusat kendali dari seluruh logika interaksi karakter, meliputi serangan biasa (attack), serangan spesial

(useSkill), dan pemulihan HP (regen). Sebelum aksi apa pun dijalankan, sistem secara disiplin akan memanggil metode isAlive() untuk memastikan bahwa karakter dengan HP 0 (berstatus *DEAD*) tidak bisa lagi beraksi. Dalam proses kalkulasi pertarungan, mekanisme pengurangan angka diatur dengan sangat hati-hati menggunakan fungsi Math.max dan Math.min. Fungsi matematika ini memastikan bahwa pukulan dengan *damage* besar tidak akan pernah membuat HP target tembus menjadi angka minus, dan sebaliknya, fitur regenerasi tidak akan membuat HP meluap melebihi kapasitas maksimal 100.

Seluruh aturan ketat tersebut kemudian dipraktikkan dan dibuktikan di dalam *class* Main yang bertindak sebagai arena arena uji coba. Di sinilah objek-objek *hero* seperti Alucard, Miya, dan Franco diciptakan untuk saling berinteraksi menggunakan jalur resmi berupa pemanggilan metode. Hasil eksekusi program di arena ini secara langsung membuktikan bahwa konsep *encapsulation* bekerja sempurna; status Franco yang awalnya dimasukkan secara tidak masuk akal berhasil dikoreksi otomatis, serangan Miya akan gagal saat *mana*-nya terkuras habis, dan pada pengujian akhir, karakter Zilong yang sudah dikalahkan terbukti tidak dapat melancarkan serangan balasan ke arah Layla.

- **Hasil Running Program**

```
=== STATUS AWAL ===
Hero: Alucard
HP: 95/100
Mana: 60/100
Damage: 12
Status: ALIVE
-----------------
Hero: Miya
HP: 80/100
Mana: 40/100
Damage: 10
Status: ALIVE
-----------------
Hero: Franco
HP: 100/100
Mana: 0/100
Damage: 15
Status: ALIVE
-----------------
```

```
=== SIMULASI BATTLE ===
Alucard menyerang Miya sebesar 12 damage. (HP Miya:80 -> 68)
Miya menggunakan skill ke Alucard sebesar 20 damage (mana -25). (HP Alucard:95 -> 75)
Franco tidak punya mana cukup untuk skill.
Miya melakukan regen +15 HP. (68 -> 83)
=== STATUS SETELAH AKSI ===
Hero: Alucard
HP: 75/100
Mana: 60/100
Damage: 12
Status: ALIVE
----------------
Hero: Miya
HP: 83/100
Mana: 15/100
Damage: 10
Hero: Miya
HP: 83/100
Mana: 15/100
Damage: 10
HP: 83/100
Mana: 15/100
Damage: 10
Mana: 15/100
Damage: 10
Damage: 10
Status: ALIVE
----------------
Hero: Franco
HP: 100/100
Mana: 0/100
Damage: 15
Status: ALIVE
----------------
```

```
=== SIMULASI BATTLE ===
Alucard menyerang Miya sebesar 12 damage. (HP Miya:80 -> 68)
Miya menggunakan skill ke Alucard sebesar 20 damage (mana -25). (HP Alucard:95 -> 75)
Franco tidak punya mana cukup untuk skill.
Miya melakukan regen +15 HP. (68 -> 83)
=== STATUS SETELAH AKSI ===
Hero: Alucard
HP: 75/100
Mana: 60/100
Damage: 12
Status: ALIVE
----------------
Hero: Miya
HP: 83/100
Mana: 15/100
Damage: 10
Hero: Miya
HP: 83/100
Mana: 15/100
Damage: 10
HP: 83/100
Mana: 15/100
Damage: 10
Mana: 15/100
Damage: 10
Damage: 10
Status: ALIVE
----------------
Hero: Franco
HP: 100/100
Mana: 0/100
Damage: 15
Status: ALIVE
----------------
```